

Lab 3-3：把"智能出行规划器"部署上云 —— CloudBase 全栈部署实践

课程：生成式软件开发

工具：Trae（推荐 Builder 模式）+ 腾讯云开发 CloudBase

前置实验：Lab 3-2（前后端分离的智能出行规划器）

提交截止：2026.05.21 23:59

提交方式：eLearning-assignment

一、实验目标

在 Lab 3-2 已经完成的"智能出行规划器"基础上，通过本次实验，你将：

1. **理解** 把一个完整的前后端分离应用"上云"意味着什么——从本地运行到任何人用手机扫码即可访问
2. **实践** 用腾讯 CloudBase 的**静态网站托管 + 云函数**替代本地开发服务器，把前端和后端都发布到公网
3. **掌握** 用 **CloudBase AI ToolKit (MCP) + CloudBase Skills** 让 AI 在 IDE 中直接帮你完成部署、查日志、管理环境变量，体验"AI 原生部署" workflow
4. **完成** 至少一项与 CloudBase 能力相关的小拓展，例如把规划数据迁移到云数据库、接入内置大模型、或实现流式 AI 回复
5. **形成** 对"本地开发 vs 云端部署"的工程化认识：部署不仅是"上传文件"，还包括安全、监控与持续演进

二、背景知识

2.1 Lab 3-2 留下的"最后一步"

Lab 3-2 结束时，你完成了一个运行在本地的前后端分离系统：

本地终端 A（后端）：`python app.py` → `http://localhost:5000`

本地终端 B（前端）：`python -m http.server` → `http://localhost:8080`

这意味着：

- 你自己可以访问，但换一台设备就打不开
- **API Key** 保存在本地 `.env`，存在忘记保护的风险
- **数据**（出行规划）保存在本地文件或内存，服务重启即消失（如果你当时偷懒用了内存存储）

本实验要做的，就是把这个系统的两个部分都迁移到 CloudBase：

前端 → CloudBase 静态网站托管（全球 CDN + HTTPS）

后端 → CloudBase 云函数（Serverless，无需维护服务器）

完成后，你的出行规划器将拥有一个 `https://...tcloudbase.com` 的公网地址，任何人用手机扫码即可访问。

2.2 为什么选 CloudBase

CloudBase 是腾讯云推出的"AI 原生一体化后端云"，对个人开发者**全部免费**：

- 新用户免费开通一个个人版环境
- 静态网站托管 + CDN + HTTPS 默认开通
- 云函数按调用次数计费，每月有充足免费额度
- 云数据库可免费存储出行规划等结构化数据
- 内置大模型 API：100 万 tokens 体验额度（可调用 DeepSeek / Hunyuan 等），无需额外申请 Key
- CloudBase AI ToolKit (MCP) + Skills：让你在 Trae/Cursor 里用自然语言完成"部署/查日志/管理函数"

对教学场景来说，最大的好处是：不需要买云服务器、不需要装 Nginx、不需要域名备案，注册后直接拿到公网 URL。

2.3 什么是 MCP？什么是 Skills？

名词	一句话解释	在本实验中的作用
MCP (Model Context Protocol)	一种让 AI 客户端（IDE）调用外部工具的标准协议	让 Trae/Cursor 里的 AI 能直接调用 CloudBase 的"部署""查日志"等操作
CloudBase AI ToolKit	CloudBase 官方提供的 MCP Server	配置在 IDE 的 <code>.trae/mcp.json</code> 或 <code>.cursor/mcp.json</code> 里
CloudBase Skills	一组预置好的"任务包"（基于 MCP 的可复用工作流）	让你能用一句"帮我把 frontend/ 部署到静态托管"完成原本要敲多条命令的事

简单理解：MCP 是协议，Skills 是基于这个协议封装好的"AI 即用脚本集"。

二、环境习炼与准备

请在动手迁移前，按顺序完成以下准备。

3.1 注册 CloudBase 并开通免费环境

1. 用微信/QQ 登录 [CloudBase 控制台](#)
2. 选择 **新建环境** → **个人版（免费）**，地域推荐选"上海"
3. 等待 1~2 分钟环境初始化完成，记下你的 **环境 ID**（形如 `your-env-xxxxxx`）

基础配置

产品类型



云开发

高可用弹性云服务，多种 Serverless 能力，高性能低代码拖拽式开发，全栈应用构建



微搭软件化

本地部署版微搭，一键安装相关依赖环境

地域

上海

环境名称

请输入环境名称

由小写字母、数字及“-”组成，且仅支持字母开头、字母数字结尾，不支持连续“-”

套餐版本

个人版

适合个人开发场景

- ✓ 资源点配额4万点
- ✓ Serverless云数据库、云存储、云函数
- ✓ 原生打通微信生态、免鉴权API调用
- ✓ 微搭可视化开发
- ✓ 接入LLM大模型、支持AI Agent开发
- ✓ CloudBase Builder自动生成应用

19.90元/月 399.00元/月

标准版

适合小团队场景

- ✓ 个人版所有内容，以及：
- ✓ 资源点配额33万点
- ✓ 更好的可用性保障（数据库回档、扩容速度）
- ✓ 对接自有数据库和CDN
- ✓ 连接微信支付
- ✓ 使用微搭高级组件和能力
- ✓ 可加入实时问题处理群

179.10元/月 1999.00元/月

企业版

适合中小型企业

- 标准版所有内容，以及：
- ✓ 资源点配额150万点
- ✓ 更加安全、稳定的独享数据库
- ✓ 企业级身份能力（组织架构、SSO、MFA等）
- ✓ 企业微信集成（登录、通讯录、可信IP）
- ✓ 审计日志和审批管理

咨询客服领专属折扣

799.20元/月 9999.00元/月

企业高级版

适合中大型企业

- 企业版所有内容，以及：
- ✓ 资源点配额650万点
- ✓ 资源配额无上限，提供定制折扣
- ✓ 可增购专家服务（压测、培训等）
- ✓ 1V1专属架构师支持（POC产品技术支持、咨询等）

立即咨询

了解更多套餐详情，请查看[价格文档](#)

个人版每月免费配额够本实验做几十次部署 + 上千次函数调用，做完实验后无需销毁环境（不用就不收费）。

3.2 安装本地工具

```
# 1) Node.js 18.15 或更高
node --version      # 期望 v18.15+

# 2) 安装 CloudBase CLI (全局)
npm install -g @cloudbase/cli

# 3) 登录 (会弹出浏览器)
tcb login
```

验证登录成功：

```
tcb env list      # 应能列出你刚创建的环境
```

3.3 在 IDE 中配置 CloudBase AI ToolKit (MCP)

Trae 用户：在你的项目根目录创建 `.trae/mcp.json`：

```
{
  "mcpServers": {
    "cloudbase": {
      "command": "npx",
      "args": [ "@cloudbase/cloudbase-mcp@latest" ],
      "env": {
        "INTEGRATION_IDE": "Trae"
      }
    }
  }
}
```

Cursor 用户：在你的项目根目录创建 `.cursor/mcp.json`：

```
{
  "mcpServers": {
    "cloudbase": {
      "command": "npx",
      "args": [ "@cloudbase/cloudbase-mcp@latest" ],
      "env": {
        "INTEGRATION_IDE": "Cursor"
      }
    }
  }
}
```

配置完成后**重启 IDE**，进入 AI 对话界面，输入：

请连接 CloudBase MCP，列出当前环境信息

如果 AI 能返回你的环境 ID、地域等信息，说明 MCP 配置成功。

3.4 安装 CloudBase Skills

在 AI 对话框里发送：

```
请帮我安装 CloudBase Skills：
执行命令 npx skills add tencentcloudbase/cloudbase-skills -y
```

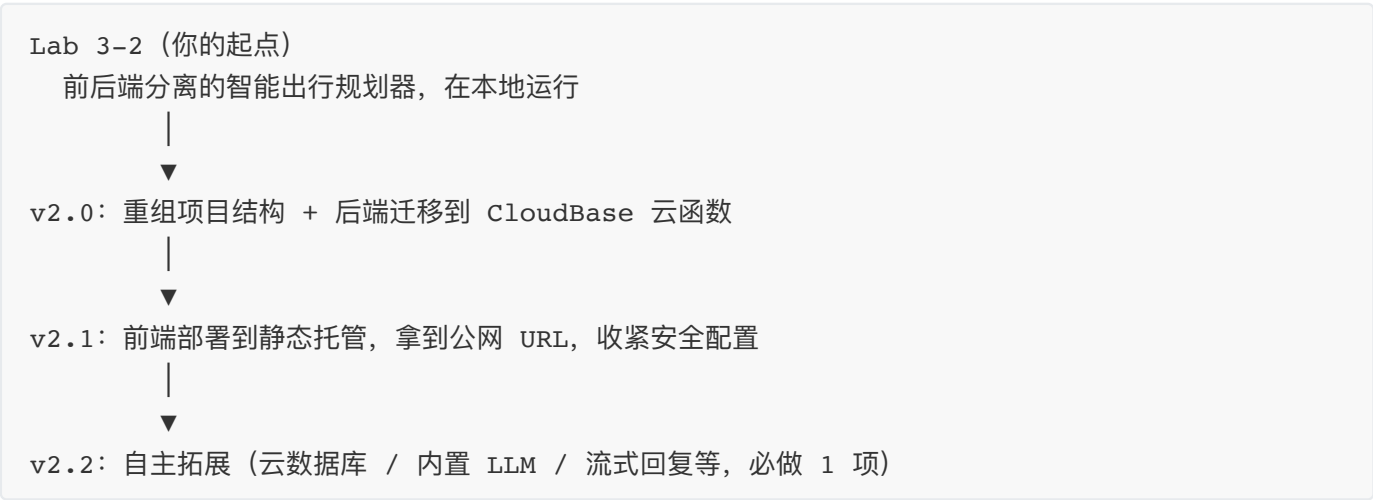
AI 会调用终端工具完成安装。安装完成后，在 AI 对话里就可以使用类似下面的"高阶指令"（请不要立即执行这条指令，只是强调有这个能力做）：

```
使用 CloudBase Skills, 把 frontend/ 部署到我的环境的静态托管
```

四、实验任务

本实验共 3 个阶段，每阶段一个核心 Git 提交。

阶段架构总览



阶段一：v2.0 重组结构 + 后端迁移到云函数

核心目标：把 Lab 3-2 的后端逻辑迁移到 CloudBase 云函数，确保前端能通过公网 URL 调用。

具体任务：

1. 重组项目结构：把 Lab 3-2 的代码整理成以下样子（让 AI 帮你做"机械搬运"是合适的，但你必须看懂每个文件的角色）：

```
project/
├── frontend/                # Lab 3-2 前端代码搬到这里
│   ├── index.html
│   ├── style.css
│   └── app.js               # 或其他框架的构建产物
└── cloudfunctions/         # 所有后端函数这里
```

```

├── cloudfunctions/           # 所有云函数在这里
│   ├── plan/               # 规划管理：新建/查看/保存/删除
│   │   ├── index.js
│   │   └── package.json
│   ├── weather/           # 天气信息
│   │   ├── index.js
│   │   └── package.json
│   └── chat/               # AI 辅助总结（如果 Lab 3-2 做了）
│       ├── index.js
│       └── package.json
├── .trae/                  # 或 .cursor/
│   └── mcp.json
├── cloudbaserc.json        # CloudBase CLI 配置 (envId 等)
├── .env.example            # 环境变量占位文件，需提交
├── .gitignore              # 必须忽略 .env、node_modules
└── README.md

```

提示：如果你的后端是 Python (Flask / FastAPI) 或 Java，云函数目前对 Node.js 支持最完善。你有两个选择：

- 用 AI 辅助把后端逻辑**改写**为 Node.js 云函数
- 或者使用 **CloudBase CloudRun**（支持 Docker 容器，可保留原语言），参见 [CloudRun 文档](#)

2. **迁移后端接口：**Lab 3-2 中的后端接口（规划管理、天气、AI 总结等）逐一迁移到对应云函数。每个云函数需处理 CORS 预检：

```

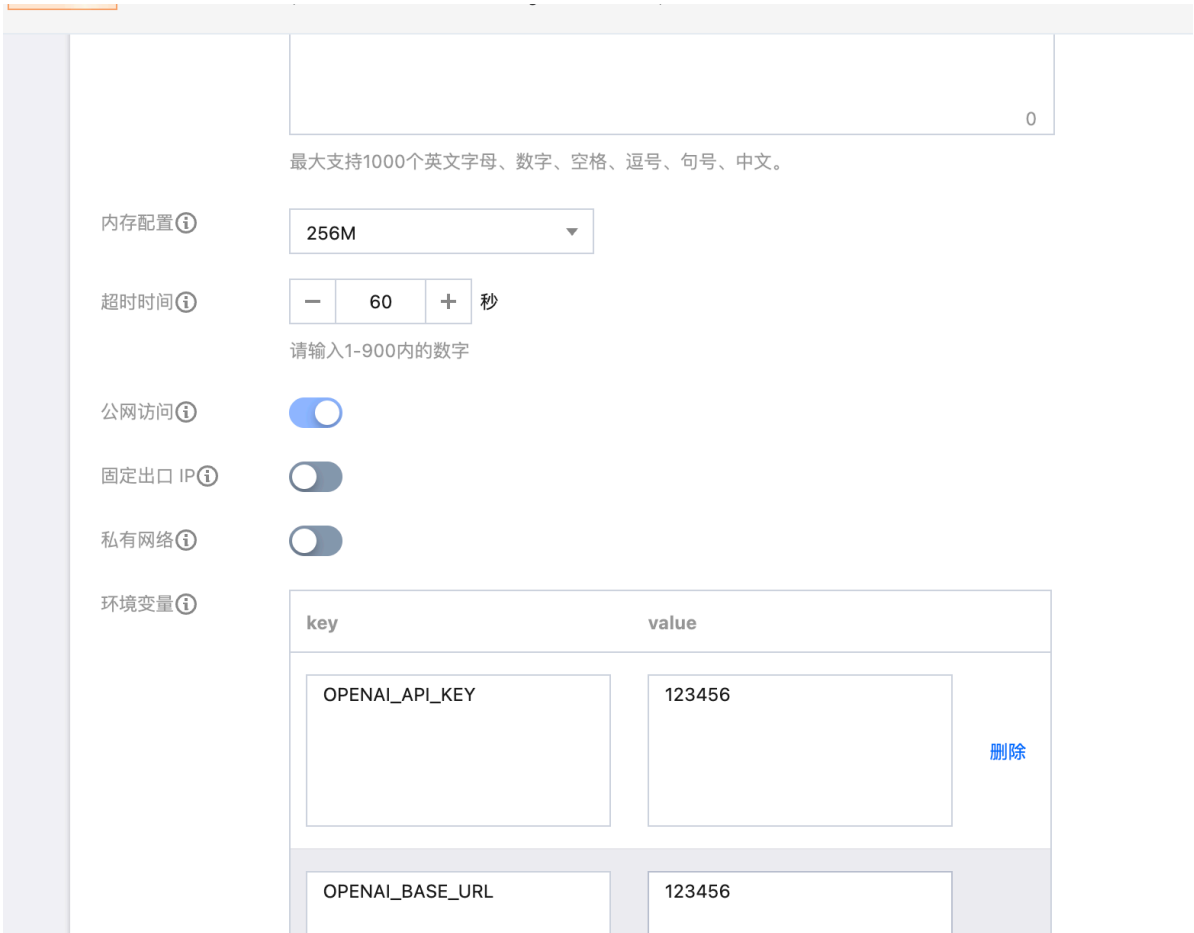
// 每个云函数的通用 CORS 处理模板
function corsHeaders() {
  return {
    'Access-Control-Allow-Origin': '*', // 阶段二会收紧到静态托管域名
    'Access-Control-Allow-Headers': 'Content-Type',
    'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS'
  };
}

exports.main = async (event, context) => {
  if (event.httpMethod === 'OPTIONS') {
    return { statusCode: 204, headers: corsHeaders(), body: '' };
  }
  // ...你的业务逻辑
};

```

3. **配置环境变量（只在云端配，不写进代码）：**

- 控制台路径：云函数 → 对应函数名 → 函数配置 → 环境变量
- 需要配置的 Key 示例：LLM API KEY、LLM BASE URL、WEATHER KEY



定时触发器

新增环境变量

删除

☒ 本地配置文件上传 ☐ 编辑代码

选择文件

请上传JSON格式的文件

HTTP访问服务路径 /ai-test

[点击配置更多规则](#)

4. 部署并配置 HTTP 触发器：

让 AI 用 CloudBase Skills 部署：

使用 CloudBase Skills 把 `cloudfunctions/` 下所有函数部署到当前环境，并为每个函数创建 HTTP 触发器

或者手动用 CLI：

```
tcb fn deploy --all
```

然后在控制台 → 云函数 → 触发管理 中为每个函数添加 HTTP 触发器

5. 更新前端调用地址**：把 `frontend/` 中所有调用本地后端（如 `http://localhost:5000`）的地址，换成对应的 CloudBase 云函数公网 URL。

阶段产出：

- 前端在本地用 `python -m http.server` 打开，可以正常调用云端后端接口
- 前端代码中不含任何 API Key

Git 提交： `git commit -m "feat: v2.0 后端迁移至 CloudBase 云函数"`

阶段二：v2.1 前端部署上线 + 安全收紧

核心目标：把前端发布到 CloudBase 静态网站托管，得到一个真实可用的公网 URL，并完成安全加固。

具体任务：

1. **部署前端。**打开 AI 对话，发送：

使用 CloudBase Skills 把 `frontend/` 目录部署到我的环境的静态网站托管

或者等价 CLI 命令：

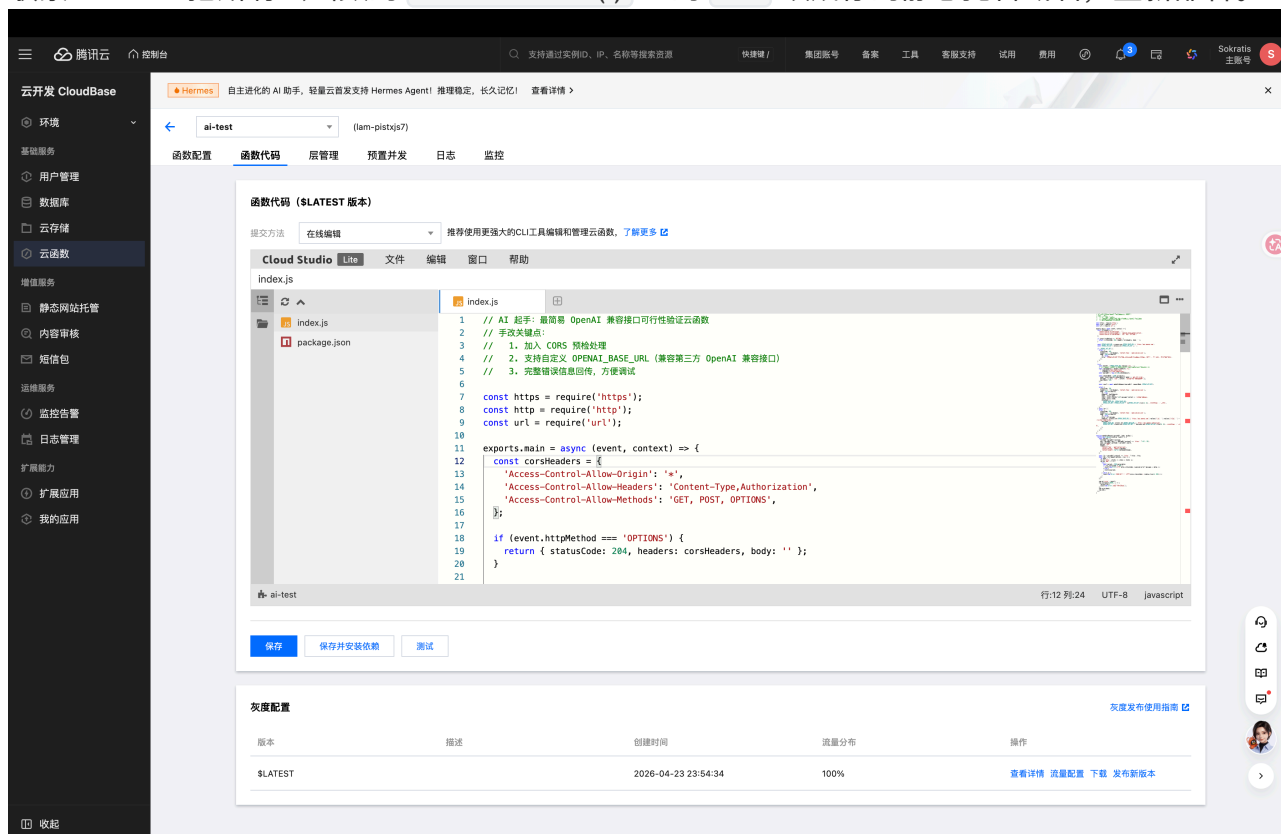
```
tcb hosting deploy ./frontend -e your-env-xxxxxx
```

如果前端使用了构建工具（Vue/React/Vite 等），先执行 `npm run build`，再把产物目录（通常是 `dist/`）部署。

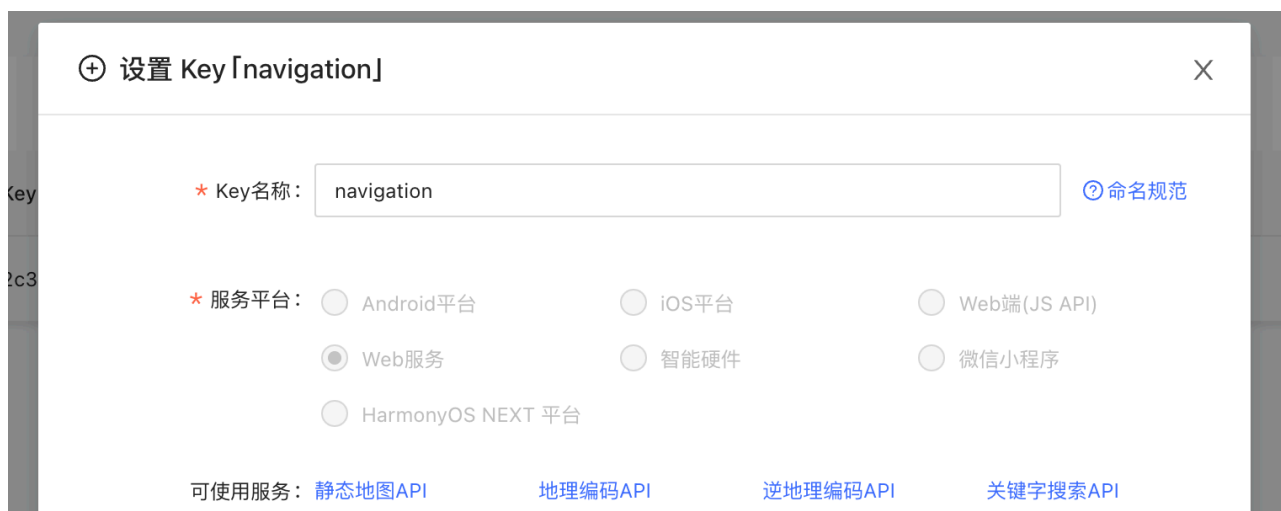
2. 获取静态托管 URL：部署成功后，在控制台 → 静态网站托管 页面找到你的访问地址，格式类似：

```
https://your-env-xxxxxx-1300000000.tcloudbaseapp.com
```

3. 收紧 CORS：把所有云函数的 `corsHeaders()` 里的 `'*'` 改成你的静态托管域名，重新部署。



4. 高德地图 Key 白名单：在高德开放平台控制台，为你的 Web 端 Key 添加静态托管域名到域名白名单。



周边搜索API

多边形搜索API

ID查询API

输入提示API

路径规划API

坐标转换API

行政区划查询API

IP定位API

天气查询API

矩形区域交通态势API

圆形区域交通态势API

指定线路交通态势API

地理围栏API

猎鹰服务API

GeoHUB服务API

IP白名单:

非必填，留空表示无IP限制

添加IP白名单后，只有白名单中的IP可访问服务

IP应该设定为服务器出口IP，支持设定IP段，如:202.202.2.*，多个IP请每行填写一条

权限: 数字签名

☐

阅读并同意 [高德地图开放平台服务协议](#) 和 [高德地图开放平台隐私权政策](#)

法人或非法人组织使用平台服务应事先购买[技术服务许可](#)以获取授权。若您未购买技术服务许可，平台向您提供的KEY和服务配额仅供您用于短期、少量的测试目的；若您超出前述范围使用平台服务（包括但不限于您开始向第三方或公众销售或提供您的应用、将您的应用于参与第三方投标、您的应用在应用程序分发平台上架或已经可以被公众获取使用、您的应用开始收费或发布广告或以其他方式获益、在组织内部使用的您的应用已上线运营、您的应用长期或大量调用平台服务等情形），构成未授权使用，平台有权采取相应措施。

取消

提交

5. **手机端验证**：用手机打开静态托管 URL，截图保存（首屏 + 一次完整的"新建规划 → 选点 → 查天气 → 保存规划"流程）。

6. **额度与监控**：在 CloudBase 控制台 → 资源监控页面截图，保存：

- 云函数调用次数
- 静态托管流量

安全检查清单（在实验报告中逐项打勾，并贴出对应证据）：

- ☐ git 仓库里搜索 `sk-`、`Bearer`、`apikey`、`KEY=` 等关键字均无真实 Key
- ☐ `.env`、`.env.local` 已加入 `.gitignore`，仓库里只有 `.env.example`
- ☐ 前端代码（`frontend/`）里只剩高德 JS API 的 Web 端 Key，且高德控制台已配置域名白名单
- ☐ 所有云函数的 CORS 已收紧到静态托管域名
- ☐ 云函数环境变量在 CloudBase 控制台配置，未硬编码进代码

阶段产出：

- 一个任何人用手机扫码即可访问的出行规划器网址
- README 中明确贴出该 URL
- 安全检查清单全部完成并附证据

Git 提交：`git commit -m "feat: v2.1 前端部署至 CloudBase 静态托管，全链路上线"`

阶段三：v2.2 自主拓展（选做 1 项）

核心目标：在已上线的全栈应用上，利用 CloudBase 的某项能力做一个小拓展，深化对"云原生能力"的理解。

参考方向（选其一，也可自主设计）：

方向	一句话提示	难度
云数据库持久化	把出行规划数据从文件/内存迁移到 CloudBase 云数据库（NoSQL），服务重启不丢数据	中
内置大模型	把 AI 辅助总结改用 CloudBase 内置的 DeepSeek / Hunyuan，无需自己管 LLM API Key	易
流式 AI 回复	云函数返回 SSE，前端用 <code>EventSource</code> 增量渲染 AI 回复，提升体验	中
历史规划列表	结合云数据库，实现跨设备查看历史规划（不同设备打开同一个 URL 能看到之前保存的规划）	中
CI/CD 自动部署	GitHub Actions + CloudBase CLI，push 即自动部署前端和云函数	中

关于"内置大模型"的彩蛋：

CloudBase 提供"零 Key"的大模型 HTTP API，可以在云函数内直接调用，无需配置外部 LLM Key：

```
// 在云函数内直接调用 CloudBase 内置 LLM (Node.js SDK)
const cloudbase = require('@cloudbase/node-sdk');
const app = cloudbase.init({ env: process.env.ENV_ID });

const { streamText } = app.AI();
const result = await streamText({
  model: 'deepseek-v3',
  messages: [{ role: 'user', content: prompt }]
});
```

可以把这个与你原来的外部 LLM Key 方式对比，体验"全托管"的差异。

自主拓展要求：

- 在报告中说明：为什么选这个方向、设计思路、AI 协作中遇到的关键偏差与修正方法
- 拓展功能必须与已上线的 v2.1 兼容，不能破坏已有体验

- 至少 1 次独立 Git 提交：`git commit -m "feat: v2.2 {你的拓展简述}"`

五、实验要求

5.1 工具与开发方式

- 云资源全部使用腾讯 CloudBase（云函数 + 静态托管，可选云数据库）
- 严格按 3 阶段递进开发，每阶段至少 1 次有意义的 Git 提交

5.2 MCP 与 Skills 使用（必须完成）

- 必须提供至少 1 张 CloudBase Skills 在 IDE 内被 AI 调用的截图（部署、查日志、初始化函数等任何场景均可）
- 在报告中至少写 1 段对 Skills/MCP 的真实评价：哪一步真的省事了？哪一步反而拖慢了？

5.3 安全与密钥管理

- 所有真实 Key 不入仓：仓库里只允许出现 `.env.example` 和 README 中"如何配置环境变量"的说明
- 云函数的 Key 必须通过 CloudBase 控制台/CLI 的"环境变量"配置，不允许硬编码到 `index.js`
- 若上传时不慎把 Key 提交到 git，必须立即在对应平台吊销该 Key 并重新申请

5.4 版本管理

- 至少 3 次核心提交，分别对应 v2.0 / v2.1 / v2.2
- 提交信息推荐格式：
 - `feat: v2.x 阶段名 + 关键功能`
 - `fix: 具体 bug 描述`
 - `chore: 部署配置 / 依赖更新`
- 最终保留完整的 `git log --oneline` 截图

六、提交内容

请打包提交以下内容，命名格式：`姓名_学号_lab3-3`

1. 项目代码：完整 `frontend/` + `cloudfunctions/` + 配置文件
 - 不要包含 `node_modules/`
 - 不要包含 `.env`，但要包含 `.env.example`
2. 公网访问 URL：写在 README 顶部，并附手机端访问截图（建议 2 张：规划创建 + AI 辅助总

- ... 文档规范 + 提交链清晰 (至少 3 张截图)
3. **CloudBase 控制台截图**：环境概览、云函数列表、本日资源监控（至少 3 张）
4. **MCP 配置**： `.trae/mcp.json`
5. **Skills 调用截图**：至少 1 张，能清晰看到 AI 通过 Skills/MCP 完成了一项 CloudBase 操作
6. **AI 协作记录**：关键 Prompt + AI 回复 + "我手改了什么"的简短标注（截图或文本均可）
7. **安全检查清单**：阶段二的清单，逐项打勾并附证据
8. `git log --oneline` **截图**：体现 v2.0 → v2.2 的完整提交链
9. **实验总结**（重点回答以下问题）：
- 把 Lab 3-2 的后端"搬上云"，遇到的最大阻力是什么？怎么解决的？
 - 与"本地运行后端"相比，CloudBase 云函数有哪些明显的好处？有哪些限制或不方便的地方？
 - Skills/MCP 在本次部署中哪些步骤真正提速了？哪些步骤反而需要你"接管"才能搞定？
 - 阶段三自主拓展时的设计思路，以及 AI 协作中遇到的关键偏差与修正方法

七、评分标准

基础评分（满分 100 分）

维度	占比	说明
阶段一：后端迁移至云函数	30%	所有接口正常工作，前端无本地后端依赖，无硬编码 Key
阶段二：前端部署上线	30%	公网 URL 真实可访问，手机端可用，CORS 与安全清单完成
阶段三：自主拓展	15%	至少 1 项拓展功能，与 v2.1 兼容，思路清晰
Skills/MCP 与标注	10%	MCP 配置 + Skills 调用截图齐全，"AI 生成 vs 手写"标注到位
版本管理与目录规范	5%	三阶段提交链清晰，目录与命名符合本文档要求
实验总结	10%	总结具体、有反思，能回答 S6 中的 4 个问题

bonus（可选，最多 +10 分）

维度	加分	说明
CI/CD 自动部署	+5	GitHub Actions 或类似工具实现 push 即部署，有 README 说明
可观测性	+3	在 README 给出云函数日志查看方式，并附一次真实调用的日志截图
多端兼容	+2	手机端 UI 有针对性优化（响应式布局），截图证明

注：若云函数无法访问、公网 URL 打不开、或仓库内出现真实 API Key，相应阶段酌情扣分。出现真实 Key 泄露的情况，建议立即吊销该 Key 并在报告中说明吊销过程。

八、提示与建议

8.1 推荐开发顺序

1. 确认 Lab 3-2 在本地能完整运行（规划、地点、天气、AI 总结）
|
2. 注册 CloudBase + 安装 CLI + 配置 MCP / Skills
|
3. v2.0：先逐个迁移云函数，每迁完一个就 curl 自测一次
|
4. v2.0：更新前端调用地址，在本地验证前端 + 云函数联调通过
|
5. v2.1：部署前端，PC 浏览器先走一遍完整流程，再扫码用手机验证
|
6. v2.1：收紧 CORS，完成安全清单
|
7. v2.2：在稳定的 v2.1 之上做拓展实验，不要边动云函数边改前端

8.2 关于后端语言的迁移

如果 Lab 3-2 的后端用了 Python 或 Java，有两条路：

路线 A：改写为 Node.js 云函数（推荐入门）

- 让 AI 帮你把 Python Flask 路由逐个翻译成 Node.js 云函数
- 重点告诉 AI："保持接口入参出参完全一致，只改语言"

路线 B：用 CloudBase CloudRun 部署容器（保留后语言）

- 写一个 `Dockerfile`，让 AI 帮你生成
- 在 CloudRun 控制台新建服务，上传代码即可
- 这个方向难度稍高，但能保留完整的 Python/Java 生态

8.3 与 AI 协作的几个"防坑"建议

- 每轮只让 AI 迁移一个函数：AI 经常"顺手重构"别的函数，造成部署失败
- 部署失败先看日志再问 AI：CloudBase 控制台 → 云函数 → 日志，把日志贴给 AI 比让它"猜"高效得多
- 环境变量要在控制台配，不要让 AI 写进代码：AI 经常"贴心"地把 Key 硬编码进 `index.js`，发现后立即手动改回 `process.env.XXX`
- CORS 问题要第一时间处理：前端从静态托管域名调用云函数时，CORS 配置不对会让所有功能都失效

8.4 常见问题速查

问题	可能原因	解决方法
云函数返回 502 / 超时	API Key 配错、上游限流、函数超时时间太短	控制台查日志；把函数超时时间调到 60s
浏览器报 CORS 错误	函数没返回 <code>Access-Control-Allow-Origin</code> ，或 OPTIONS 没单独处理	用本文档中的 <code>corsHeaders()</code> 模板
前端调接口报 404	HTTP 触发器路径没配 / 路径写错	控制台 → 云函数 → 触发管理 → 重新确认路径
部署后页面访问 404	<code>index.html</code> 不在上传目录的根层	确认 <code>frontend/index.html</code> 存在且在部署目录的根部
MCP 在 IDE 里不出现工具	Node 版本太低 / 没重启 IDE	升级到 Node 18.15+，完全退出 IDE 后重启

如有疑问，请在课程群中提问，或在实验课时向助教咨询。